Standard Softmax Attention Supports In-Context Policy Improvement

Anonymous Authors

Keywords: in-context reinforcement learning; softmax attention; policy improvement; SARSA; Expected SARSA; approximate greedification

Abstract

Can standard normalised softmax attention support policy improvement in-context? Recent constructive work establishes policy improvement for linear attention and fixed-policy temporal-difference (TD) evaluation for standard softmax, but does not connect normalised softmax to action-value control. We give a positive answer using state–action value tokens and signed softmax-kernel write-back.

Our first construction starts from raw transition and persistent Q-memory tokens. In a precisely specified two-block residual attention–FFN network, two retrieval heads read the current and sampled next-pair values, a fixed ReLU map forms the signed sampled-SARSA residual, and a state–action head writes it to Q-memory. The exact identity uses externally supplied, input-dependent equality-routing masks and an external visited-query/null-token gate; it recovers the simultaneous per-pair mean tabular SARSA update. Removing the equality masks gives a finite-logit, gate-assisted approximation for which we bound retrieval and write-back leakage separately. Coupled to a policy derived from the current action values, the exact singleton construction yields the usual sequential on-policy control loop. Our second construction adds an action-attention stage. Its finite-temperature target is exactly the conditional Expected-SARSA target of the current Boltzmann policy and simultaneously approximates greedy action selection. For action sharpness β, it obeys

$$0 \leq \max_a Q(s,a) - \sum_a \mathrm{softmax}(\beta Q(s,.))_a Q(s,a) \leq \log(|A|)/\beta.$$

Combining this bound with Bellman-optimality contraction yields an explicit neighbourhood guarantee for the ideal tabular approximate-greedy update. Q-retrieval leakage, residual write-back leakage and finite-context error enter as separate perturbations. These guarantees strengthen, but are not required for, the sampled-SARSA control route.

An executable literal matrix witness verifies the complete prompt, all projections, masks, residual states and write-back against both a compact implementation and the tabular reference. Sep-

arately, controlled tabular experiments evaluate policy behaviour using compact operators; the large sampled-control sweep directly indexes the two Q-values and therefore is not an execution of the full prompt network. Across 30 sampled-control tasks, this compact loop raises exact mean policy return by 0.429 and gives positive final $J_m u$ gain on 30/30 tasks while closely tracking a shared-trajectory exact-match comparator. The two-stage operator gives positive final $J_m u$ gain on 20/20 expected-update tasks and approaches exact greedy control as action attention sharpens. The result establishes an oracle-routed fixed-weight softmax-attention realisation of tabular action-value control, without claiming learned routing, statewise dominance or monotonic improvement of every stochastic update.

1. Introduction

Can standard normalised softmax attention support policy improvement in-context? Fixed-weight Transformers can execute learning algorithms in their forward pass [1–3], and recent work gives constructive reinforcement-learning examples. Liang and Lai [4] show that linear attention can implement parametric semi-gradient SARSA and actor–critic updates. Xie et al. [5] show that standard softmax attention can implement weighted TD policy evaluation for a fixed policy. Together these two lines leave open whether the latter, normalised attention mechanism can support action-value control rather than evaluation alone.

Policy improvement does not require a maximum in every TD target. SARSA forms the on-policy residual

$$\delta_k^S = r_k + \gamma Q(s'_k, a'_k) - Q(s_k, a_k)$$

using the action a'_k actually selected by the current behaviour policy. Updating Q and then deriving the next behaviour policy from the updated values gives an on-policy control loop [6]. A greedy Bellman backup is a different, stronger route: it places action selection inside the target through $max_b Q(s', b)$. The two routes answer complementary questions and should not be conflated.

We represent each state–action pair x=(s,a) by a persistent Q-memory token that stores Q(x), while a raw transition token contains current- and next-pair identifiers and reward. Two fixed masked heads retrieve the two Q-values required by sampled SARSA, a position-wise map forms the signed residual, and a final standard-softmax head performs scalar write-back. This tokenisation avoids updating a global parameter vector through a sample-dependent product of δ and ϕ. The resulting write-back is

$$Q^+(x) = Q(x) + \alpha \sum_k K_\eta(x_k, x)\delta_k,$$

where K_η is a normalised state–action matching kernel and the signed TD residual is carried by the value vectors. Positive attention weights therefore do not remove negative TD corrections.

Our first construction proves the full raw-token computation rather than assuming sampled SARSA residuals as inputs. It is a precisely specified two-block residual attention–FFN network with

standard exponential softmax, bidirectional structured attention, and no LayerNorm or dropout; it is not a decoder-only causal Transformer. Externally supplied, input-dependent equality-routing masks retrieve Q(s,a) and Q(s',a'), a fixed two-unit ReLU map forms the residual, and an external visited-query/null-token gate completes the simultaneous per-pair mean tabular SARSA write-back. The singleton version generates the usual sequential SARSA iterates; coupling those iterates to a behaviour policy gives the standard on-policy control loop under the usual external environment and sampling interface. When the equality masks are removed, finite logits approximate retrieval and write-back with an explicit one-step bound while the external visited-query gate remains.

Our second construction adds an action-attention stage. For the current Boltzmann policy

$$p_\beta(b|s) = \exp(\beta Q(s,b))/\sum_c \exp(\beta Q(s,c)),$$

the stage returns

$$M_\beta Q(s) = \sum_b p_\beta(b|s) Q(s,b).$$

At finite β this is exactly the conditional Expected-SARSA target of the current Boltzmann policy. It also approximates greedy action selection, with the uniform bound

$$0 \le \max_b Q(s,b) - M_\beta Q(s) \le \log(|A|)/\beta.$$

The Expected-SARSA and approximate-greedy interpretations are both valid. We use the second interpretation only to derive a stronger optimal-control consequence: in the ideal tabular, synchronous setting, the resulting update is a bounded perturbation of relaxed Bellman optimality. The proof uses the contraction of $T^\star$, not an unproved contraction property of the smooth operator. Kernel mismatch and finite-context error enter as separate perturbations.

Controlled tabular experiments test both control routes through policy behaviour. For scale, the sampled-SARSA sweep uses a compact implementation in which direct table indexing supplies the two Q-values and softmax implements the write-back kernel; it does not execute the literal prompt network. It is compared with a shared-trajectory exact-match write-back comparator, which isolates finite softmax matching without being mislabelled as an independent on-policy learner. A separate literal matrix witness tests the complete prompt construction. The two-stage compact operator is compared with exact greedy control while action sharpness and kernel sharpness are varied. These experiments are designed to answer whether the corresponding softmax operators improve policies.

Our contributions are:

1. We give an end-to-end fixed-weight standard-softmax construction that retrieves current action values from Q-memory tokens, forms sampled-SARSA residuals and writes them back, and we separate its exact externally routed identity from its equality-mask-free but gate-assisted finite-logit approximation.

2. We show that the two-stage finite-temperature target is exactly Boltzmann Expected SARSA and simultaneously a uniformly bounded approximation to greedy action selection.
3. We derive action-value and policy-performance guarantees for the stronger approximate-greedy route, with explicit action-sharpness, kernel and finite-context terms.
4. We evaluate both routes through matched control experiments and policy return.

The result is constructive and deliberately scoped. Exact content selection uses externally supplied, input-dependent structured equality masks and an external visited-query/null-token gate. Finite equality-mask-free logits remain gate-assisted and are approximate. Environment interaction, exact $\varepsilon - greedy$ argmax and random action sampling remain external. We do not claim that every stochastic TD step monotonically improves a policy, that a generic pretrained Transformer automatically learns the proposed operators or routing, or that the main control sweep executes the full token network. The main guarantees concern finite tabular problems with explicit coverage conditions.

2. Related Work

Transformers as in-context algorithms. Linear and self-attention constructions have been connected to gradient descent, ridge regression and other learning algorithms executed within a forward pass [1–3]. This constructive viewpoint motivates asking which reinforcement-learning control operations can be represented by a fixed attention mechanism.

Constructive in-context reinforcement learning. Liang and Lai [4] provide the closest policy-improvement result. Their linear-attention construction updates a global parameter vector through a semi-gradient SARSA term, and their actor–critic construction provides a second on-policy route. Xie et al. [5] instead use standard normalised softmax attention to implement non-parametric TD policy evaluation for a fixed Markov reward process. Our state–action token construction follows the signed softmax write-back mechanism of [5], but couples it to action-value control. It differs from [4] in both attention type and representation: normalised softmax updates Q-tokens directly, while linear attention forms the parametric semi-gradient product needed to update a global weight vector.

SARSA, Expected SARSA and greedy control. Classical SARSA is an on-policy control method when its action-value updates are coupled to a behaviour policy derived from the current estimates [6]. Expected SARSA replaces the sampled next action value by its conditional expectation under the target policy. For the Boltzmann policy induced by the current Q-values, that conditional expectation is exactly the softmax-weighted action value used by our two-stage operator. Greedy Bellman control is not required for policy improvement, but it supplies a direct route to $Q^\star$ and a policy-performance bound. We therefore treat sampled SARSA and approximate greedification as complementary softmax control constructions.

Normalisation and action selection. A normalised softmax head returns a convex combination of its value vectors [7]. This does not remove signed coordinates already carried by those values, so a softmax kernel can aggregate positive and negative TD residuals. Normalisation does introduce a quantitative sharpness trade-off when attention is used to choose among actions. The item-count

sharpness analysis of Velickovic et al. [8] is relevant when actions are treated as the candidate items in our approximate-greedy route.

Scope. The contribution is an operator-level existence result for policy improvement with standard softmax attention. It neither reproduces linear-attention training dynamics nor claims that a pretrained model necessarily discovers the construction. The sampled-SARSA route establishes the direct on-policy mechanism; the two-stage route strengthens it by placing approximate greedification inside the forward operator.

3. End-to-End Construction and Guarantees

3.1 Exact model class and complete prompt

Let $\mathcal{X} = \mathcal{S} \times \mathcal{A}$, let $m = |\mathcal{X}|$, and fix an enumeration of $\mathcal{X}$. Pair x has one-hot identifier $e_x \in \mathbb{R}^m$. One update receives a frozen table Q_l, fixed scalars $\gamma \in [0, 1)$ and $\alpha > 0$, and N sampled transitions

$$(S_k, A_k, R_k, S'_k, A'_k), \qquad k = 1, \dots, N,$$

where A'_k is the next action actually selected by the current behaviour policy. Write $X_k = (S_k, A_k)$, $X'_k = (S'_k, A'_k)$ and $n_x = \sum_{k=1}^{N} \mathbf{1}\{X_k = x\}$.

We prove the result for a precisely specified residual attention network. It has two attention–FFN blocks, standard exponential softmax, no LayerNorm and no dropout. Tokens are columns. It uses bidirectional structured attention and is not a decoder-only causal Transformer. The projection matrices are fixed once m, N, γ, α are fixed. The structured equality-routing masks are externally supplied and input-dependent. An external support gate supplies the visited-query/null-token rule from the discrete pair identifiers in the sampled batch. Thus the theorem is an exact oracle-routed reference construction. It does not prove or assert that a generic pretrained Transformer learns or discovers routing.

Take token width $d = 2m + 8$ and the direct-sum convention

$$\mathbb{R}^d = \underbrace{\mathbb{R}^3}_{\text{type}} \oplus \underbrace{\mathbb{R}^m}_{\text{current ID}} \oplus \underbrace{\mathbb{R}^m}_{\text{next ID}} \oplus \underbrace{\mathbb{R}^5}_{(r,q,u,v,\delta)}.$$

Let $\tau_Q, \tau_T, \tau_Z \in \mathbb{R}^3$ be the three type basis vectors. The scalar coordinates store reward r, persistent memory q, retrieved current value u, retrieved next value v, and TD residual δ. Define

$$\begin{aligned} M_x &= (\tau_Q, e_x, 0_m, 0, Q_l(x), 0, 0, 0)^\top, \\ T_k &= (\tau_T, e_{X_k}, e_{X'_k}, R_k, 0, 0, 0, 0)^\top, \\ Z &= (\tau_Z, 0_m, 0_m, 0, 0, 0, 0, 0)^\top, \\ H^{(0)} &= [M_{x_1}, \dots, M_{x_m}, T_1, \dots, T_N, Z] \in \mathbb{R}^{d \times L}, \qquad L = m + N + 1. \end{aligned} \tag{3.1}$$

The null token Z is not the zero vector because it carries its type coordinate, but every coordinate read as an attention value below is zero. In particular, a raw transition token contains neither $Q_l(X_k)$, $Q_l(X'_k)$ nor a precomputed residual.

Let $P_c, P_n \in \mathbb{R}^{m\times d}$ select the current- and next-ID blocks. Let $\mathbf{e}_r, \mathbf{e}_q, \mathbf{e}_u, \mathbf{e}_v, \mathbf{e}_\delta \in \mathbb{R}^d$ be basis columns for the five scalar coordinates. For a head with $W_Q, W_K \in \mathbb{R}^{d_h\times d}$, $W_V \in \mathbb{R}^{d_v\times d}$ and $W_O \in \mathbb{R}^{d\times d_v}$, define query-by-source weights

$$A_{ij} = \frac{\exp\big((W_Q h_i)^\top (W_K h_j)/\sqrt{d_h} + \mathcal{M}_{ij}\big)}{\sum_{s=1}^{L} \exp\big((W_Q h_i)^\top (W_K h_s)/\sqrt{d_h} + \mathcal{M}_{is}\big)}.$$

In matrix form the head output is $W_O(W_V H)A^\top \in \mathbb{R}^{d\times L}$. Every mask row below has nonempty support. Non-target queries are routed to Z, so no row is an all-$-\infty$ softmax.

Theorem 3.1 (exact sampled-SARSA under structured equality routing). For fixed m, N, γ, α and the supplied equality-routing masks and visited/null support rule below, the two-block residual attention–FFN network maps the raw prompt in (3.1) to Q-memory satisfying, for every $x \in \mathcal{X}$,

$$Q_{l+1}(x) = \begin{cases} Q_l(x) + \dfrac{\alpha}{n_x} \displaystyle\sum_{k:X_k=x} [R_k + \gamma Q_l(X'_k) - Q_l(X_k)], & n_x > 0, \\ Q_l(x), & n_x = 0. \end{cases}$$

3.2 Proof of Theorem 3.1: retrieval and residual formation

Step 1: current-pair retrieval. The first head uses

$$\begin{gathered} W_Q^{\mathrm{cur}} = P_c, \qquad W_K^{\mathrm{cur}} = P_c, \qquad W_V^{\mathrm{cur}} = \mathbf{e}_q^\top, \qquad W_O^{\mathrm{cur}} = \mathbf{e}_u, \\ \mathcal{M}_{ij}^{\mathrm{cur}} = 0 \iff (i = T_k,\ j = M_{X_k}) \text{ or } (i \notin \{T_1, \dots, T_N\},\ j = Z). \end{gathered} \tag{3.2}$$

All other mask entries are $-\infty$. For transition query T_k, the only admitted source is M_{X_k}. Hence its softmax row has weight one at that source, independently of the numerical logit, and

$$W_O^{\mathrm{cur}} \sum_j A_{T_k j}^{\mathrm{cur}} W_V^{\mathrm{cur}} h_j = \mathbf{e}_u\, \mathbf{e}_q^\top M_{X_k} = Q_l(X_k)\mathbf{e}_u.$$

A Q-memory or null query admits only Z; since $\mathbf{e}_q^\top Z = 0$, its head output is zero.

Step 2: next-pair retrieval. The second head is parallel to the first and reads the next-ID block only at its query:

$$\begin{gathered} W_Q^{\mathrm{next}} = P_n, \qquad W_K^{\mathrm{next}} = P_c, \qquad W_V^{\mathrm{next}} = \mathbf{e}_q^\top, \qquad W_O^{\mathrm{next}} = \mathbf{e}_v, \\ \mathcal{M}_{ij}^{\mathrm{next}} = 0 \iff (i = T_k,\ j = M_{X'_k}) \text{ or } (i \notin \{T_1, \dots, T_N\},\ j = Z), \\ W_O^{\mathrm{next}} \sum_j A_{T_k j}^{\mathrm{next}} W_V^{\mathrm{next}} h_j = Q_l(X'_k)\mathbf{e}_v. \end{gathered} \tag{3.3}$$

Again all unspecified entries are $-\infty$, and every non-transition query receives zero from the head.

Both heads read the same $H^{(0)}$, as required for parallel multi-head attention. Their output projections write to disjoint coordinates, so the residual state after the attention sublayer is

$$\begin{aligned} H^{(a)} &= H^{(0)} + O^{\mathrm{cur}} + O^{\mathrm{next}}, \\ M_x^{(a)} &= M_x, \qquad Z^{(a)} = Z, \\ T_k^{(a)} &= (\tau_T, e_{X_k}, e_{X'_k}, R_k, 0, Q_l(X_k), Q_l(X'_k), 0)^\top. \end{aligned}$$

Step 3: exact position-wise residual map. Set

$$\begin{aligned} g^\top = \mathbf{e}_r^\top + \gamma \mathbf{e}_v^\top - \mathbf{e}_u^\top, \qquad W_1 = \begin{bmatrix} g^\top \\ -g^\top \end{bmatrix} \in \mathbb{R}^{2\times d}, \\ W_2 = \mathbf{e}_\delta \begin{bmatrix} 1 & -1 \end{bmatrix} \in \mathbb{R}^{d\times 2}, \qquad W_2\,\mathrm{ReLU}(W_1 h) = \mathbf{e}_\delta g^\top h. \end{aligned} \tag{3.4}$$

The last identity follows from $\mathrm{ReLU}(z) - \mathrm{ReLU}(-z) = z$. Consequently the first block's FFN residual gives

$$H^{(1)} = H^{(a)} + W_2\,\mathrm{ReLU}(W_1 H^{(a)}), \qquad \delta_k^S = R_k + \gamma Q_l(X'_k) - Q_l(X_k).$$

For Q-memory and null tokens, the r, u, v coordinates are zero, so the FFN output is zero. Thus Block I preserves every identifier, reward and Q-memory value, while it writes exactly the two retrieved values and the signed sampled-SARSA residual to each transition token.

3.3 Proof of Theorem 3.1: write-back and assembly

Step 4: complete write-back head. The second block has one active head:

$$\begin{aligned} &W_Q^{\mathrm{wr}} = P_c, \qquad W_K^{\mathrm{wr}} = P_c, \qquad W_V^{\mathrm{wr}} = \mathbf{e}_\delta^\top, \qquad W_O^{\mathrm{wr}} = \alpha \mathbf{e}_q, \\ &\mathcal{M}_{ij}^{\mathrm{wr}} = 0 \quad \Longleftrightarrow \quad \begin{cases} i = M_x,\ j = T_k,\ X_k = x, & n_x > 0, \\ i = M_x,\ j = Z, & n_x = 0, \\ i \in \{T_1, \dots, T_N, Z\},\ j = Z. \end{cases} \end{aligned} \tag{3.5}$$

All other entries are $-\infty$. For a visited memory query M_x and every admitted source T_k,

$$\frac{(P_c M_x)^\top (P_c T_k)}{\sqrt{m}} = \frac{e_x^\top e_{X_k}}{\sqrt{m}} = \frac{1}{\sqrt{m}}.$$

The n_x admitted logits are therefore equal, so their weights are exactly $1/n_x$. Since the value projection selects the signed scalar δ_k^S, the head output at M_x is

$$\frac{\alpha}{n_x} \sum_{k: X_k = x} \delta_k^S\, \mathbf{e}_q.$$

Positive softmax weights do not remove negative TD corrections: the sign resides in the value coordinate, not in the attention weight. If $n_x = 0$, the only source is Z and $\mathbf{e}_\delta^\top Z = 0$, so the output is zero. Transition and null queries also read only Z and remain unchanged.

Step 5: assemble the two blocks. Set every unused projection block to zero and set the second block's FFN to zero. Relative to the initial states in (3.1), the complete token-state changes are

token	after Block I attention	after Block I FFN	after Block II
M_x	unchanged	unchanged	set $q = Q_{l+1}(x)$
T_k	set $u = Q_l(X_k)$ and $v = Q_l(X'_k)$	set $\delta = \delta_k^S$	unchanged
Z	unchanged	unchanged	unchanged

Every coordinate not listed in the table remains exactly as it was in $H^{(0)}$.

Reading the Q-memory coordinate of $H^{(2)}$ gives

$$Q_{l+1}(x) = \begin{cases} Q_l(x) + \dfrac{\alpha}{n_x} \displaystyle\sum_{k:X_k=x} \left[R_k + \gamma Q_l(X'_k) - Q_l(X_k)\right], & n_x > 0, \\ Q_l(x), & n_x = 0. \end{cases} \tag{3.6}$$

This is exactly the theorem statement. Every displayed matrix is a fixed coordinate selector or fixed scalar multiple for the declared m, N, γ, α; all data dependence is confined to the prompt and supplied masks. Both Q-lookups, residual formation and Q write-back are therefore included in the two-block computation. □

Exact scope of the identity. Equation (3.6) is a simultaneous per-pair mean update at frozen Q_l. It is not the global-batch semi-gradient convention

$$Q_l(x) + \frac{\alpha}{N} \sum_{k:X_k=x} \delta_k^S,$$

and repeated occurrences of a pair are not processed sequentially within a frozen batch. A singleton call has $n_x = 1$ and does recover the usual sequential tabular SARSA step. The construction requires input-dependent equality routing and external visited/null support detection; neither is a causal mask learned by softmax. It does not show that a generic pretrained Transformer discovers the layout. The exact network is fixed for fixed m, N, γ, α; a varying Robbins–Monro step size requires the corresponding family of output projections or an additional multiplication mechanism. At each control round we rebuild the prompt, or equivalently clear and reinitialize the u, v, δ scratch coordinates. Section 3.8 removes the equality masks at finite logits but still retains the visited-query gate.

3.4 From an update operator to control

Corollary 3.2 (sequential SARSA equivalence). If one sampled transition is processed per control round and only its visited pair receives a non-null write-back query, the construction instantiated with step size α_t produces

$$\begin{aligned} &Q_{t+1}(S_t, A_t) \\ &= Q_t(S_t, A_t) \\ &+\alpha_t[R_{t+1} + \gamma Q_t(S_{t+1}, A_{t+1}) - Q_t(S_t, A_t)], \end{aligned}$$

with all other state–action values unchanged. A sequence of blocks whose fixed output scales are the prescribed α_t therefore generates exactly the tabular sequential sampled-SARSA iterates. A single fixed block corresponds to a constant step size.

For a finite tabular MDP with bounded rewards, the sequential construction inherits the classical almost-sure convergence result under infinite state–action visitation, per-pair Robbins–Monro step sizes and a greedy-in-the-limit with infinite exploration (GLIE) policy sequence [6]. The frozen mini-batch experiment in Section 5.2 uses a different update convention and fixed exploration, so that convergence theorem is not asserted for the experimental protocol.

An individual SARSA residual is not a policy-improvement theorem. The usual exact policy-improvement bridge is logically separate. If π is ε-soft, Q^π has been evaluated exactly, and π' is ε-greedy with respect to Q^π, then writing $\pi = \varepsilon/|\mathcal{A}| + (1-\varepsilon)\tilde{\pi}$ gives

$$\begin{aligned} &\sum_a \pi'(a|s) Q^\pi(s,a) \\ &= \frac{\varepsilon}{|\mathcal{A}|} \sum_a Q^\pi(s,a) \\ &+(1-\varepsilon) \max_a Q^\pi(s,a) \\ &\geq V^\pi(s). \end{aligned}$$

Hence $V^{\pi'} \geq V^\pi$ by the policy-improvement theorem. With an estimate $\widehat{Q}$ satisfying $\|\widehat{Q} - Q^\pi\|_\infty \leq \xi$, the worst-case statement weakens to $V^{\pi'} \geq V^\pi - 2\xi/(1-\gamma)$; monotonicity requires a sufficient advantage margin. Theorem 3.1 establishes representational equivalence, Corollary 3.2 supplies a conditional online convergence route, and the frozen-batch experiment supplies empirical before–after policy evidence. These are three distinct claims.

3.5 Internal Boltzmann action attention

The sampled construction conditions on the next action A'_k supplied by the behaviour interface. Standard softmax can also form a Boltzmann behaviour distribution internally. For $\beta > 0$, group the Q-memory tokens by state and use score $\beta Q_l(s,b)$ for action token b. The action head returns

$$p_\beta(b|s;Q_l) = \frac{\exp(\beta Q_l(s,b))}{\sum_c \exp(\beta Q_l(s,c))}.$$

An external sampler may draw $A' \sim p_\beta(\cdot|S';Q_l)$ and append it to a raw transition, after which Theorem 3.1 implements sampled SARSA. Alternatively, using $Q_l(s,b)$ as the action-head values gives the internal conditional expectation

$$M_\beta Q_l(s) = \sum_b p_\beta(b|s;Q_l) Q_l(s,b).$$

The associated conditional Bellman target is

$$(T_\beta Q)(s,a) = \mathbb{E}[R + \gamma M_\beta Q(S') \mid s,a].$$

Proposition 3.3 (Expected-SARSA identity). If A' conditional on S' is drawn from $p_\beta(\cdot|S';Q)$, then

$$\mathbb{E}[R + \gamma Q(S',A') \mid s,a] = (T_\beta Q)(s,a).$$

Proof. Conditional on S', the inner expectation over A' is $\sum_b p_\beta(b|S';Q)Q(S',b) = M_\beta Q(S')$. Taking the remaining conditional expectation proves the identity. □

At finite β, this is exactly the conditional Expected-SARSA operator for the Boltzmann policy tied to the current Q-values. It can also be analysed as approximate greedification; finite β is not exact Q-learning.

3.6 Softmax approximation of greedy action selection

Lemma 3.4 (entropy bound for softmax action aggregation). For every $q \in \mathbb{R}^d$ and $\beta > 0$,

$$0 \le \max_a q_a - \sum_a \mathrm{softmax}(\beta q)_a q_a \le \frac{\log d}{\beta}.$$

Proof. Let $p = \mathrm{softmax}(\beta q)$ and define $\mathrm{LSE}_\beta(q) = \beta^{-1} \log \sum_a \exp(\beta q_a)$. The Gibbs entropy identity gives $\mathrm{LSE}_\beta(q) = \sum_a p_a q_a + H(p)/\beta$. Since $\max_a q_a \le \mathrm{LSE}_\beta(q)$, $\sum_a p_a q_a \le \max_a q_a$ and $0 \le H(p) \le \log d$, the result follows. □

Corollary 3.5 (greedy-target deviation). For every Q,

$$\begin{aligned}&\|T_\beta Q - T^\star Q\|_\infty \\ &\le \frac{\gamma \log |\mathcal{A}|}{\beta},\end{aligned}$$

where

$$(T^\star Q)(s,a) = \mathbb{E}[R + \gamma \max_b Q(S', b) \mid s, a].$$

3.7 Stronger guarantee for the approximate-greedy route

Consider the synchronous full-coverage update with exact state–action matching,

$$Q_{l+1} = (1-\alpha)Q_l + \alpha T_\beta Q_l,$$

where $0 < \alpha \le 1$, and define $\rho = 1 - \alpha(1-\gamma)$.

Theorem 3.6 (bounded perturbation of Bellman optimality). The iterates satisfy

$$\begin{aligned}&\|Q_{l+1} - Q^\star\|_\infty \\ &\le \rho \|Q_l - Q^\star\|_\infty \\ &+ \frac{\alpha\gamma \log |\mathcal{A}|}{\beta}.\end{aligned}$$

Consequently,

$$\begin{aligned}&\limsup_{l\to\infty} \|Q_l - Q^\star\|_\infty \\ &\le \frac{\gamma \log |\mathcal{A}|}{\beta(1-\gamma)}.\end{aligned}$$

Proof. Add and subtract $T^\star Q_l$. The relaxed identity contributes $1-\alpha$, the contraction of $T^\star$ contributes $\alpha\gamma$, and Corollary 3.5 contributes the additive target deviation. Iterating the resulting scalar recurrence gives the limit-superior bound. The proof does not assume that T_β itself is a contraction. □

If $\beta_l \to \infty$ with depth, the additive term tends to zero and the same stable scalar recurrence yields convergence to $Q^\star$. Fixed β gives a controlled neighbourhood rather than exact convergence.

3.8 Finite-logit deviations without equality masks

The exact theorem delegates pair equality to masks. We now remove the two retrieval masks and the write-back equality mask, while retaining the external visited-query gate. The result is therefore equality-mask-free but gate-assisted.

For retrieval sharpness $\zeta > 0$, scale the one-hot projections so that the correct Q-memory logit is ζ and every incorrect logit is zero. Softmax is now taken over all m Q-memory tokens. Its correct mass, total off-target mass, and induced residual error obey

$$\begin{gathered} p_\zeta = \frac{e^\zeta}{e^\zeta + m - 1}, \qquad \lambda_R = 1 - p_\zeta = \frac{m-1}{e^\zeta + m - 1}, \\ |\widetilde{Q}_l(x) - Q_l(x)| \le \lambda_R \operatorname{span}(Q_l), \\ |\tilde{\delta}_k^S - \delta_k^S| \le (1+\gamma)\lambda_R \operatorname{span}(Q_l) =: E_R, \end{gathered} \tag{3.7}$$

where $\operatorname{span}(Q_l) = \max_x Q_l(x) - \min_x Q_l(x)$. The first value bound holds because a probability mass λ_R is moved from the correct table entry to values lying within the table span. The residual uses one current and one discounted next retrieval, producing the factor $1 + \gamma$.

For write-back sharpness $\eta > 0$, use logit $\eta\langle e_{X_k}, e_x\rangle$. For a visited query x, the finite attention kernel and the exact group-mean kernel are

$$\begin{gathered} K_\eta(k|x) = \frac{\exp(\eta \mathbf{1}\{X_k = x\})}{n_x e^\eta + N - n_x}, \qquad U(k|x) = \frac{\mathbf{1}\{X_k = x\}}{n_x}, \\ \varepsilon_W(x) := \sum_{k=1}^N |K_\eta(k|x) - U(k|x)| = \frac{2(N - n_x)}{n_x e^\eta + N - n_x}. \end{gathered} \tag{3.8}$$

To verify the last identity, matching tokens contribute total absolute difference $(N - n_x)/(n_x e^\eta + N - n_x)$ and nonmatching tokens contribute the same amount.

Proposition 3.7 (finite-logit end-to-end update error). Suppose $|\delta_k^S| \le B$ for every transition. For every visited query,

$$|Q_{l+1}^{\text{soft}}(x) - Q_{l+1}^{\text{exact}}(x)| \le \alpha[E_R + B\varepsilon_W(x)]. \tag{3.9}$$

Proof. Write the finite update with approximate residuals as $\alpha \sum_k K_\eta(k|x)\tilde{\delta}_k^S$ and the exact update as $\alpha \sum_k U(k|x)\delta_k^S$. Add and subtract $\alpha \sum_k K_\eta(k|x)\delta_k^S$. Since $K_\eta(\cdot|x)$ is a probability vector, (3.7) bounds the residual-replacement term by αE_R. Since $|\delta_k^S| \le B$, (3.8) bounds the kernel-replacement term by $\alpha B \varepsilon_W(x)$. Summing the two bounds proves (3.9). □

For a frozen policy and frozen Q_l, define the finite-context residual deviation

$$\varepsilon_D = \max_{x: n_x > 0} \left| \frac{1}{n_x} \sum_{k: X_k = x} \delta_k^S - [(T^\pi Q_l)(x) - Q_l(x)] \right|.$$

The implemented update differs from the corresponding visited population update by at most $\alpha(E_R+B\varepsilon_W+\varepsilon_D)$. In the approximate-greedy full-coverage recurrence, these terms enter additively:

$$\|Q_{l+1}-Q^\star\|_\infty \le \rho\|Q_l-Q^\star\|_\infty + \alpha\left[\frac{\gamma\log|\mathcal{A}|}{\beta}+E_R+B\varepsilon_W+\varepsilon_D\right].$$

This is a one-step operator-deviation statement. It does not turn an arbitrary finite trajectory into a global Bellman recursion, and it does not supply a concentration rate without an independent-sampling, mixing or martingale assumption. Retrieval sharpness, write-back sharpness and action sharpness control three different operations.

3.9 Policy consequence

Corollary 3.8. If $\|Q-Q^\star\|_\infty \le \epsilon$ and π_Q is greedy with respect to Q, then

$$\begin{aligned}&\|V^\star - V^{\pi_Q}\|_\infty \\ &\le \frac{2\epsilon}{1-\gamma}.\end{aligned}$$

Thus the approximate-greedy action-value guarantee implies a conditional policy-performance guarantee. The sampled-SARSA route instead obtains exact operator equivalence from Theorem 3.1, a conditional asymptotic result from Corollary 3.2, and empirical final-return evidence under the separate frozen-batch protocol.

4. Method

4.1 End-to-end construction and compact experimental operator

Section 3 gives the full token computation in a literal $d \times L$ prompt. Raw transition tokens contain pair identifiers and rewards; two structured-routing retrieval heads copy the required current Q-values from Q-memory tokens; an explicit two-unit ReLU FFN forms the signed TD residual; and a final state–action head writes it back. All $W_Q, W_K, W_V, W_O, W_1, W_2$ matrices are fixed coordinate maps. No residual or current Q-value is supplied as an initial transition-token field.

For efficiency, the large control sweep bypasses the two prompt-level retrieval heads: direct table indexing supplies the same two scalars, the same fixed linear expression forms the residual, and the standard-softmax kernel performs the write-back. It therefore tests the compact write-back/control operator, not execution of the full prompt network. A separate literal matrix witness instantiates the complete prompt, every projection, all query-by-source routing masks, the null-token gate, residual scratch states and the write-back head, then compares the result with both the compact implementation and the tabular reference. The experiments use one-hot state–action identifiers, so neither route tests learned representations or function approximation.

For a visited query x, the compact finite-logit write-back uses

$$K_\eta(k|x) = \frac{\exp(\eta\langle\psi(X_k), \psi(x)\rangle)}{\sum_j \exp(\eta\langle\psi(X_j), \psi(x)\rangle)}$$

and writes

$$\begin{aligned} &Q_{l+1}(x) \\ &= Q_l(x) + \alpha_l \sum_k K_\eta(k|x)\delta_k. \end{aligned}$$

The attention weights are positive and normalised, but δ_k is signed; negative TD corrections therefore remain available. Queries absent from the context receive no update. The exact construction in Theorem 3.1 uses externally supplied equality-routing masks and an externally gated zero-value null token for unvisited queries. Removing the equality masks gives the explicit finite-logit approximation analysed jointly with finite-logit Q retrieval in Section 3.8; that approximation still retains the visited-query gate.

4.2 Route I: sampled SARSA control

For each batch, draw A'_k from the current behaviour policy at S'_k. The two retrieval heads and the fixed residual map of Section 3 form

$$\begin{aligned} &\delta_k^S \\ &= R_k + \gamma Q_l(S'_k, A'_k) - Q_l(X_k). \end{aligned}$$

The state–action head then performs the write-back above. Under exact matching this is precisely

$$\begin{aligned} &Q_{l+1}(x) \\ &= Q_l(x) + \frac{\alpha_l}{n_x} \\ &\sum_{k:X_k=x} \delta_k^S, \end{aligned}$$

the simultaneous per-pair mean mini-batch SARSA update at frozen Q_l. With a single queried transition it is the usual sampled SARSA update. It is not a summed update over repeated visits and not a sequential update in which Q changes within the batch.

To close the control loop, the next behaviour policy is derived from Q_{l+1}. We use an ε-greedy policy in the sampled experiment. The TD target itself contains no maximum: the next action is the action sampled from the current policy. In the theorem and literal witness, Q retrieval, residual formation and Q write-back occur in the constructed blocks. In the large control sweep, retrieval is replaced by direct table indexing and only the compact residual/write-back kernel is run. Environment interaction, exact ε-greedy argmax and random action sampling are external

interfaces. Theorem 3.1 is an operator identity, Corollary 3.2 concerns the different singleton online protocol, and the present frozen-batch protocol is evaluated empirically rather than assigned either theorem's conclusion.

4.3 Route II: Boltzmann Expected SARSA and approximate greedification

The stronger route adds a standard softmax head over the action tokens at each next state. Its scores are $\beta Q_l(S'_k, b)$, its values are $Q_l(S'_k, b)$, and a group mask restricts attention to actions sharing S'_k. The result is

$$M_\beta Q_l(S'_k) = \sum_b \frac{\exp(\beta Q_l(S'_k, b))}{\sum_c \exp(\beta Q_l(S'_k, c))} Q_l(S'_k, b).$$

The next linear projection forms

$$\begin{aligned} &\delta_k^\beta \\ &= R_k + \gamma M_\beta Q_l(S'_k) - Q_l(X_k), \end{aligned}$$

and the common state–action head writes this residual to Q-tokens. No exact maximum is used inside the proposed operator. At finite β the target is the conditional Expected-SARSA target for the current Boltzmann policy. Viewed instead as an approximation to greedy control, its error is bounded by $\gamma \log(|A|)/\beta$ at the Bellman-target level, yielding Theorem 3.6.

4.4 Scope of the construction

The exact sampled-SARSA result concerns a two-block residual attention–FFN network with standard exponential normalisation, fixed coordinate projections, bidirectional structured attention, no LayerNorm and no dropout; it is not a decoder-only causal Transformer. Its matrices are fixed for fixed m, N, γ, α. It starts from raw transitions and persistent Q-memory tokens and does not assume a precomputed TD residual. Exact retrieval and write-back rely on externally supplied, input-dependent equality-routing masks and an external visited-query/null-token gate. Proposition 3.7 removes the equality masks but retains the gate. The claims do not assert that a generic pretrained Transformer discovers the layout or learns routing, that finite logits perform exact content selection, or that simulator action execution and random sampling occur inside a deterministic attention head. A variable step-size schedule requires a family of blocks with different fixed output scales or an additional multiplication mechanism.

5. Experiments

5.1 Questions, environments, and policy metric

The experiments ask three questions tied directly to the central claim:

1. Does the compact one-stage softmax write-back operator match its sampled-SARSA reference, and does the separate literal witness match that compact operator?
2. Does closing that operator with its behaviour policy improve policy return?
3. Does the two-stage operator retain policy improvement while its internal action aggregation approaches greedy control?

Each task is a finite random MDP. Transition rows and the initial-state distribution are sampled from symmetric Dirichlet distributions, and transition rewards are uniform on [-1,1]. For a policy π and initial distribution μ, we compute the discounted return exactly,

$$J_\mu(\pi) = \mu^\top (I - \gamma P_\pi)^{-1} r_\pi,$$

so policy comparisons contain no rollout-evaluation noise. Unless noted otherwise, reported dispersions are sample standard deviations across independently sampled tasks.

5.2 Sampled SARSA: matched write-back and closed-loop improvement

We use 30 MDPs with 9 states, 4 actions, and $\gamma = 0.5$. The standard-softmax learner starts from a small random Q-table and uses an $\varepsilon = 0.1$-greedy behaviour policy derived from its current Q. At each of 200 control updates, it generates a 128-transition trajectory, freezes that batch's Q and policy, and applies the per-pair mean sampled-SARSA update with $\eta = 12$ and

$$\alpha_l = 0.35/\sqrt{1 + 0.02(l-1)}.$$

The matched reference receives the same transitions and step sizes but uses the exact-match kernel U. It isolates the effect of finite softmax matching; it is not a second independently sampled control run.

Table 1. Exact policy return for sampled-SARSA control (30 MDPs, mean with sample standard deviation in parentheses).

update rule	initial return	final return	return gain	positive $J_m u$ gain
standard-softmax SARSA	0.104 (0.211)	0.533 (0.117)	0.429 (0.190)	30/30
exact-match write-back	0.104 (0.211)	0.532 (0.117)	0.428 (0.192)	30/30

The two-sided Student-t 95% confidence interval for the softmax learner's mean gain is [0.358,0.500]. Its greedy-policy return rises from 0.110 to 0.587, compared with an optimal return of 0.621. At the final update, its Q-table differs from the shared-trajectory comparator by only 2.19e-4 in infinity norm (SD 5.27e-5), and their greedy policies agree on 0.996 of states (SD 0.020). A same-frozen-Q

one-step check has mean discrepancy 1.54e-4; every checked step lies below its finite-kernel bound. The largest observed non-matching attention mass is 7.80e-4. These paired controlled results show positive final $J_m u$ gain on every sampled task and operator fidelity, not statewise policy dominance or monotonicity of every stochastic SARSA step. Indeed, intermediate evaluated returns sometimes decrease.

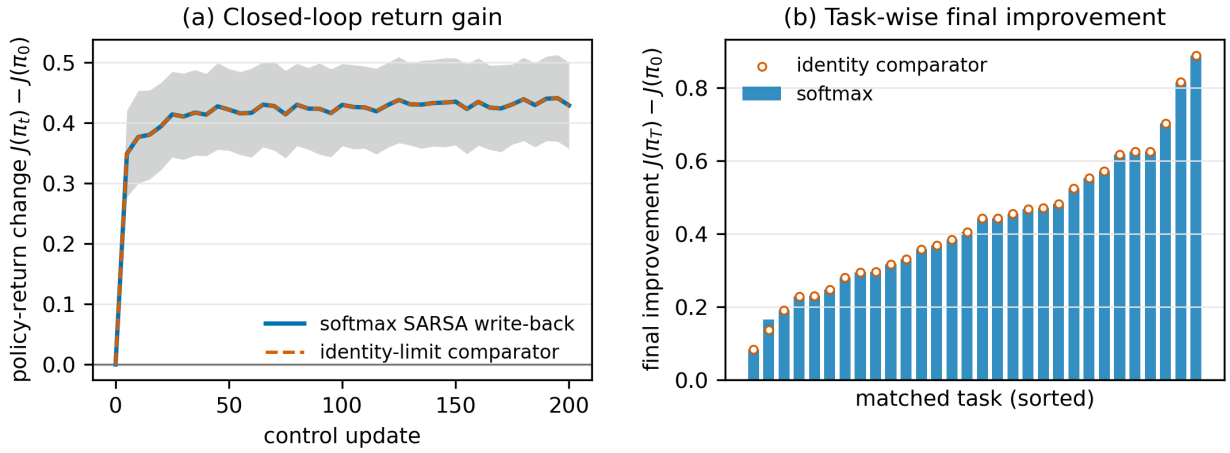

Figure 1. Standard-softmax sampled SARSA gives positive final policy-return gain. Left: mean exact return change across control updates; shading is a 95% confidence interval. Right: final return gain in every matched task, with the shared-trajectory identity comparator overlaid.

5.3 Two-stage route: policy gain and action sharpness

The second experiment isolates internal action aggregation from sampling noise. On 20 new MDPs with the same state, action, and discount sizes, every state–action pair is updated from its exact conditional expectation. We start from $Q_0 = 0$, use $\alpha = 0.5$ and state–action sharpness $\eta = 20$, and run 200 synchronous updates. Greedy argmax ties, including the initial all-zero tie, select the lowest-index action. The exact-max kernel operator is an oracle reference; the proposed operator uses β in {1,2,5,10,20}. Final policies are greedy with respect to the resulting Q-tables using the same rule.

The mean initial policy return is -0.0569 (SD 0.262), while the oracle reaches 0.5209 (SD 0.151), a gain of 0.5777 (SD 0.224). Every softmax temperature gives positive final $J_m u$ gain on all 20 tasks.

Table 2. Two-stage softmax control after 200 expected updates (means over 20 MDPs).

operator	return gain	optimal-return gap	Q infinity error	greedy agreement
exact-max oracle	0.57774	0	7.48e-13	1.000
softmax, $\beta = 1$	0.57682	9.25e-4	0.2465	0.972
softmax, $\beta = 2$	0.57766	7.51e-5	0.1905	0.989

operator	return gain	optimal-return gap	Q infinity error	greedy agreement
softmax, $\beta = 5$	0.57766	7.51e-5	0.08758	0.989
softmax, $\beta = 10$	0.57774	0	0.03441	1.000
softmax, $\beta = 20$	0.57773	8.98e-6	0.01131	0.994

The action-value error falls as β increases. The observed statewise softmax-to-max gap falls from 0.3618 at $\beta = 1$ to 0.02007 at $\beta = 20$, below the corresponding bounds 1.386 and 0.06931. Greedy policies are piecewise constant in Q, so policy return can already match the oracle before the action-value error vanishes; the table therefore reports both quantities rather than treating Q error as a substitute for policy improvement.

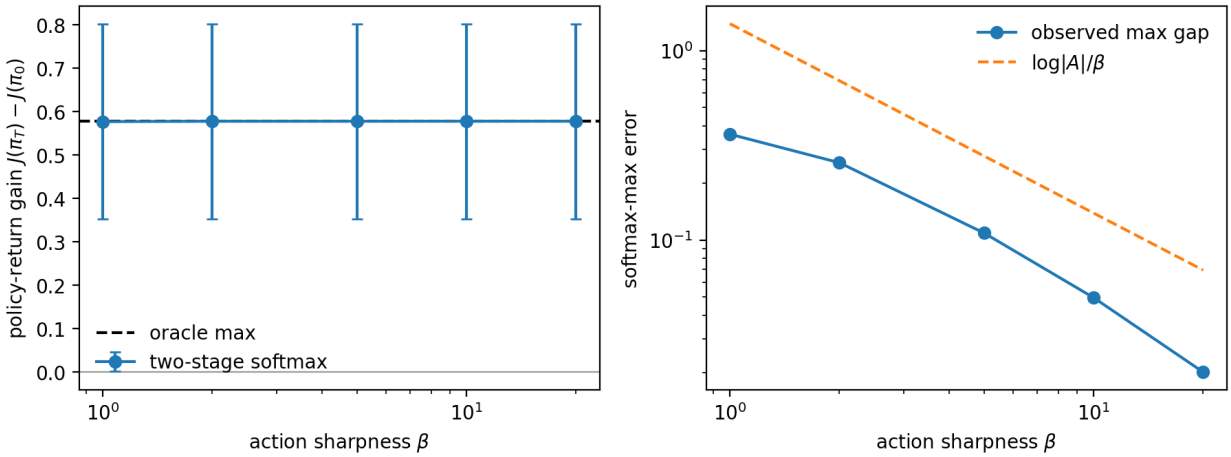

Figure 2. The two-stage operator gives positive policy-return gain and approaches greedy action selection. Left: mean return gain with sample-standard-deviation bars; the dashed line is the exact-max oracle. Right: the observed softmax-to-max gap remains below $\log(|A|)/\beta$.

5.4 End-to-end and finite-sharpness checks

The literal matrix witness uses a 32×20 prompt on the deterministic fixture and instantiates every stated projection and full query-by-source mask. Its masks have nonempty support, all softmax rows are finite and normalised, retrieved values and signed residual coordinates match the intended token states, and unvisited queries route to the zero-value null token. Its final Q update differs from both the compact implementation and the explicit frozen-batch tabular reference by 5.55×10^{-17}; six successive singleton updates also match sequential tabular SARSA. Eight unvisited pairs are unchanged. This is an implementation witness for the complete construction, distinct from the large control sweep. With deliberately low retrieval and write-back sharpness $(\zeta, \eta) = (2.4, 2.1)$, the observed retrieval error is 0.425 below its 0.999 bound, and the complete end-to-end update error is 0.250 below the Proposition 3.7 bound 1.170.

The earlier compact direct formula check gives exact-match per-pair write-back to 1.11×10^{-16}. At the control experiment's $\eta = 12$, the observed synthetic one-step finite-kernel error is 8.55×10^{-5}, below its computed bound 2.00×10^{-4}. In the two-stage experiment's 20-update sharpness sweep, reducing non-matching mass from 0.928 at $\eta = 1$ to 0.00159 at $\eta = 10$ reduces mean Q error from 0.547 to 0.01158; $\eta = 20$ changes it only to 0.01158. These checks support the finite-step retrieval and write-back distortion analysis. They are not used to claim that leakage is an unavoidable asymptotic error floor.

6. Discussion

6.1 Direct answer to the policy-improvement question

Yes, standard normalised softmax attention can support policy improvement in the precise oracle-routed constructive sense established here. In Route I, two equality-routed retrieval heads read the current and sampled next-pair values from Q-memory, a fixed ReLU map forms the signed SARSA residual, and a final matching head performs the per-pair mean write-back. The exact identity assumes the external masks and visited/null gate. Equality-mask-free finite-logit retrieval and write-back approximate their exact counterparts while retaining the gate. Closing the external operator–policy loop gives positive final J_μ gain in the controlled tasks, whose large sweep uses the compact direct-indexing implementation. In Route II, an action-softmax head supplies the current Boltzmann expected continuation value and provides a controlled approximation to greedy action selection. The empirical claim rests on before-versus-after J_μ, not on decreasing TD error or statewise dominance.

This answer does not require Q-learning. SARSA already becomes a control method when its behaviour policy is updated from its current action values [6]. The maximum-based route is retained only because it proves that standard softmax can also carry out internal approximate greedification and because Bellman optimality supplies a stronger neighbourhood guarantee.

6.2 Relation to linear-attention SARSA

Liang and Lai [4] use linear attention to update a shared parameter vector, which requires a sample-dependent product between a TD residual and a feature vector. Our construction stores Q directly at state–action memory tokens. Standard softmax heads retrieve the two required scalar values and later route the resulting signed residual back to the matching memory token; no dynamic residual–feature product is required. The representations differ, but both implement SARSA control updates. Relative to fixed-policy softmax TD constructions [5], persistent state–action memory, explicit Q retrieval and closure of the behaviour-policy loop are the essential changes.

6.3 Limitations

The theory is tabular and assumes one persistent memory token per state–action pair. Exact retrieval and write-back use externally supplied, input-dependent structured equality masks and an external visited-query/null-token gate; finite equality-mask-free logits are approximate and remain gate-assisted. The exact network omits LayerNorm and dropout and is not a decoder-only

causal Transformer. The sampled experiment freezes Q and the behaviour policy within each batch, directly indexes the two Q-values and uses per-pair mean updates; classical online SARSA convergence instead applies to singleton sequential updates under GLIE and per-pair Robbins–Monro conditions. A fixed block supplies a fixed step size, so a variable schedule requires a block family or another multiplication mechanism. The two-stage expected experiment has full coverage and isolates operator bias rather than finite-trajectory concentration. Exact $\varepsilon-greedy$ argmax, random policy sampling and environment action execution remain outside the attention blocks. Extending the construction to learned representations must address approximate identifiers, learned or mask-free routing, support gating, coverage and persistent-memory scaling.

7. Conclusion

Standard normalised softmax attention can support in-context policy improvement. The essential mechanism is signed state–action value write-back: attention weights perform contextual matching, while positive and negative TD corrections remain in the value vectors.

The sampled-SARSA construction gives the direct on-policy route. Starting from raw transition identifiers and rewards, two externally equality-routed softmax heads retrieve the current and sampled next-pair values from persistent Q-memory tokens, an explicit two-unit ReLU map forms the signed residual, and a state–action head writes it to the visited Q-memory tokens. An external visited-query/null-token gate handles absent pairs. The singleton construction generates the usual sequential SARSA iterates. Removing the equality masks gives finite-logit retrieval and write-back approximations with separate error terms, while retaining that gate. Coupling the updated values to the next behaviour policy closes the external interaction loop without requiring a maximum in the TD target.

The two-stage construction gives a stronger internal action-selection result. Its finite-temperature action aggregation is the expected continuation value of the current Boltzmann policy; adding reward and discount produces the conditional Expected-SARSA target. The same continuation value approaches the greedy value as action attention sharpens, with error at most $\log(|A|)/\beta$. This deviation, combined with Bellman-optimality contraction, yields a neighbourhood guarantee around $Q^\star$ and a corresponding greedy-policy performance bound in the ideal tabular setting.

The experiments evaluate both routes through policy behaviour. The large one-stage sweep uses direct table indexing for the two retrieved Q-values and tests the compact residual/write-back control operator against a shared-trajectory exact-match comparator; it is not a run of the complete prompt network. A separate literal matrix witness verifies the full prompt, projections, routing masks, residual states and write-back. The two-stage compact operator is tested against exact greedy control under varying action and kernel sharpness. Together these results show final policy-return gains through sampled on-policy updates and, with an additional stage, through approximate internal greedification.

The exact result remains an oracle-routed, fixed-weight token-level construction for fixed problem size and update parameters. Its two-block residual attention–FFN network has no LayerNorm or dropout, uses bidirectional structured attention rather than decoder-only causal attention,

and relies on externally supplied input-dependent equality-routing masks plus an external visited-query/null-token gate. It does not place environment simulation or random action draws inside deterministic attention, establish per-update monotonic improvement under stochastic sampling, show automatic routing discovery by a pretrained Transformer, or give guarantees beyond finite tabular problems with the stated coverage conditions. Extending the same mechanisms to learned representations, learned or mask-free exact routing and finite-trajectory coverage is the main open direction.

References

1. J. von Oswald, E. Niklasson, E. Randazzo, J. Sacramento, A. Mordvintsev, A. Zhmoginov, and M. Vladymyrov. Transformers Learn In-Context by Gradient Descent. ICML, 2023. arXiv:2212.07677.
2. E. Akyurek, D. Schuurmans, J. Andreas, T. Ma, and D. Zhou. What Learning Algorithm Is In-Context Learning? Investigations with Linear Models. ICLR, 2023. arXiv:2211.15661.
3. S. Garg, D. Tsipras, P. Liang, and G. Valiant. What Can Transformers Learn In-Context? A Case Study of Simple Function Classes. NeurIPS, 2022. arXiv:2208.01066.
4. H. Liang and L. Lai. Transformers Provably Implement In-Context Reinforcement Learning with Policy Improvement. 2026. arXiv:2605.05755.
5. Z. Xie, X. Liu, C. Chen, S. D. Liu, R. Chandra, and S. Zhang. Beyond Linear Attention: Softmax Transformers Implement In-Context Reinforcement Learning. 2026. arXiv:2605.07333.
6. R. S. Sutton and A. G. Barto. Reinforcement Learning: An Introduction. Second edition, MIT Press, 2018.
7. O. Richter and R. Wattenhofer. Normalized Attention Without Probability Cage. ICLR, 2021. arXiv:2005.09561.
8. P. Velickovic, C. Perivolaropoulos, F. Barbero, and R. Pascanu. Softmax Is Not Enough (for Sharp Size Generalisation). ICLR, 2025. arXiv:2410.01104.